# SIMATS ENGINEERING

## CO5-ASSESSMENT TOOL3-Reverse Engineering

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1709**

| | |
|---|---|
| **Register Number** | **192511061** |
| **Name** | **Vidhushini.T.S** |

---

**COURSE OUTCOME COVERED**

***CO5:*** *Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)*

*Assessment Tool Weightage: 20%*

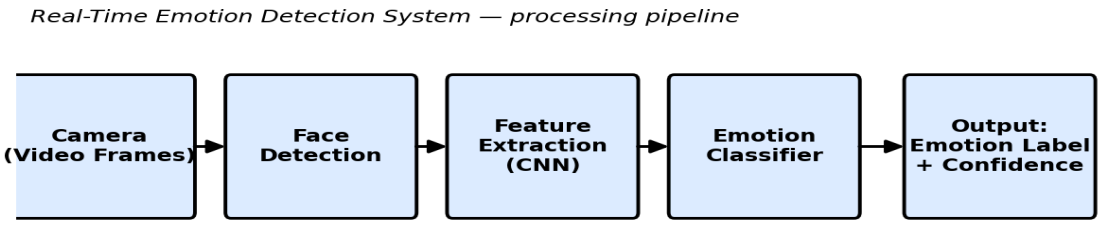
**QUESTIONS: 5** **Total Marks:50**

---

**Q1.** The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

| Speaker | Message |
|---|---|
| User | I want to book a flight to Chennai next Friday. |
| Chatbot | Sure! What time would you like to depart, and do you have a preferred airline? |
| User | Morning flight, no preference. |
| Chatbot | I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first? |

**Q2.** The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d)




classifying the agent type (e.g., simple reflex, model-based) with justification.

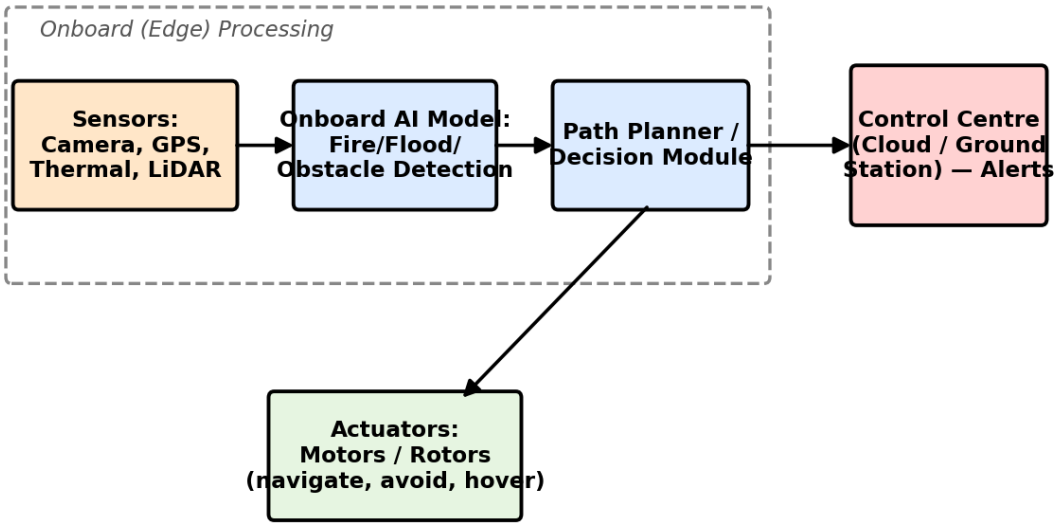
**Q3.** An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

| Rule | IF (condition) | THEN (conclusion) |
|---|---|---|
| R1 | fever = yes AND cough = yes | suspect = flu |
| R2 | suspect = flu AND body_ache = yes | diagnosis = Influenza |
| R3 | fever = yes AND rash = yes | suspect = measles |

**Given facts:** fever = yes, cough = yes, body_ache = yes
**Reasoning trace:** R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.
***Explanation generated by the shell:*** *"Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza."*

**Q4.** The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

*Autonomous Drone — Disaster Detection (Edge AI) system architecture*



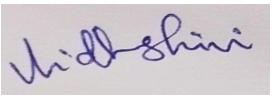

**Q5.** The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final

Real/Fake decision may have been derived from the frame-level confidence scores, and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

| Frame No. | Blink Rate (per 10s) | Texture/Artifact Score | Fake Confidence (%) | Prediction |
|---|---|---|---|---|
| 1 | 4 | 0.12 | 8 | Real |
| 2 | 1 | 0.61 | 74 | Fake |
| 3 | 0 | 0.83 | 91 | Fake |
| 4 | 5 | 0.09 | 5 | Real |
| 5 | 2 | 0.55 | 68 | Fake |

**Overall system verdict:** Fake (3 of 5 frames flagged — majority vote)
***Code of Conduct:***

*I Vidhushini T.S 192511061 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

**RUBRICS FOR CALCULATION**

| Criteria | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning | Max Marks |
|---|---|---|---|---|---|
| Identification of Agent Type & PEAS (correctly inferred from the given artifact) | Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact. | Identifies the agent type and PEAS description correctly with only minor omissions. | Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact. | Agent type/PEAS identification is largely incorrect or not attempted. | **10** |
| Identification of Architecture & Components (mapping artifact evidence to system components) | Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact. | Identifies most components correctly with reasonable mapping to the artifact; minor gaps present. | Identifies some components; mapping to the artifact is weak or partially incorrect. | Little or no correct identification of architecture/components. | **10** |
| Identification of Underlying | Correctly reverse-enginee | Correctly identifies | Identification is only | Technique/mechanism is | **10** |

| Criteria | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning | Max Marks |
|---|---|---|---|---|---|
| Technique/Algorithm/Reasoning Mechanism | rs the technique, algorithm, or reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence. | the technique/ mechanism with adequate, mostly accurate justification. | partially correct or lacks sufficient supporting evidence. | incorrectly identified or not addressed. | |
| Quality of Justification & Use of Evidence (linking conclusions back to the given artifact) | Every conclusion is clearly justified with specific, accurate references to the given artifact. | Most conclusions are justified with reasonable references to the artifact. | Justification is vague or only loosely connected to the artifact. | Conclusions are stated with little or no justification or evidence. | **10** |
| Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner) | All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner. | Most sub-parts are addressed clearly with minor gaps in organisation or completeness. | Some sub-parts are missing or the presentation is unclear/poorly organised. | Response is incomplete, disorganised, or largely missing. | **10** |
| | | | | Total | 50 |

**Question 1: AI Flight-Booking Chatbot**

(a) PEAS Description

- Performance Measure: High user satisfaction, high accuracy in parameter extraction (destination, date, time preference), minimal turns/dialogue depth to complete booking, low cost/optimal flight retrieval, zero booking errors.
- Environment: Dynamic flight booking database/API, natural language text interface, varied user inputs and intent states.
- Actuators: Screen text interface (displaying options, queries, and summaries), API function calls to query flight databases or reserve seats.
- Sensors: Text input field/keyboard receiving user prompts, natural language stream.

(b) Agent Classification & Justification

- Classification: Goal-Based Agent (with sequential dialogue state tracking).
- Justification: The chatbot does not merely react to the latest input. It maintains an internal state (slots to fill: *Destination* = Chennai, *Date* = Next Friday, *Time* = Morning, *Airline* = No preference) to achieve a explicit goal: successfully searching/booking a compliant flight. It chooses actions (asking clarifying questions or offering options) based on how close it is to filling all required parameters for the goal state.

(c) Component Identification for Each Response

1. User: *"I want to book a flight to Chennai next Friday."*
   Chatbot Response: *"Sure! What time would you like to depart, and do you have a preferred airline?"*

   Component Responsible: Dialogue Manager (DM) (utilizing NLU slot extractions).

   *Explanation:* The NLU parses parameters (*Destination*: Chennai, *Date*: Next Friday). The DM evaluates the goal slot frame, identifies missing slots (*Time*, *Airline*), decides the dialog act (request_info), and hands off the response frame to NLG to render the prompt.
2. User: *"Morning flight, no preference."*
   Chatbot Response: *"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"*

   Component Responsible: Knowledge Base / Database Interface combined with Dialogue Manager (DM) & NLG.

   *Explanation:* NLU updates slots (*Time*: Morning, *Airline*: None). DM triggers a query against the Knowledge Base (Flight DB API) which returns 3 results. DM then decides the next dialog state (confirm_sorting_preference), rendered into human-readable text by the NLG (Natural Language Generation) component.

(d) Enhancement using a Learning-Agent Architecture

- Enhancement: Integration of a Reinforcement Learning from Human Feedback (RLHF) or Q-learning based Dialogue Policy Manager.
- Mechanism: Rather than following hardcoded slot-filling rules, the learning element treats dialog management as a Markov Decision Process (MDP). It learns optimal dialogue paths based on reward signals (e.g., successful booking = +10 reward, extra turn required = -1 penalty, user abandonment = -10 penalty). Over time, it learns personalized user strategies (e.g., automatically offering the cheapest morning option first to price-sensitive users without asking, shortening dialogue turns).

**Question 2: Real-Time Emotion Detection System**

(a) Pipeline Stages & Functions

1. Stage 1: Frame Acquisition & Image Preprocessing
   *Function:* Captures raw image/video frames from camera feeds, normalizes brightness/contrast, resizes dimensions, and crops noise.
2. Stage 2: Face Detection & Bounding Box Localization
   *Function:* Scans the frame to locate human faces, bounding the region of interest (ROI) containing spatial facial regions.
3. Stage 3: Facial Landmark Detection & Feature Extraction
   *Function:* Maps key structural points on the face (eyebrows, eyes, nose, mouth contour) and extracts geometric/textural facial descriptors.
4. Stage 4: Emotion Classification & Confidence Scoring
   *Function:* Maps extracted landmark spatial configurations to target emotion categories (e.g., Happy, Sad, Angry, Surprised, Neutral) and assigns confidence probabilities.

(b) AI/ML Techniques Used at Each Stage

- Stage 1 (Preprocessing): Histogram Equalization, Gaussian Smoothing, Image Rescaling algorithms.
- Stage 2 (Face Detection): Convolutional Neural Networks (CNNs) like Single Shot MultiBox Detector (SSD), MTCNN, or Haar Cascades / OpenCV HOG.
- Stage 3 (Landmark Extraction): Dlib Ensemble of Regression Trees (ERT), MediaPipe Face Mesh, or Multi-task CNNs.
- Stage 4 (Emotion Classification): Deep Convolutional Neural Networks (ResNet/VGG Architecture), Support Vector Machines (SVM), or Vision Transformers (ViT).

(c) PEAS Description

- Performance Measure: High Classification Accuracy / F1-Score across varied lighting/demographics, real-time processing rate ($\geq 30\ FPS$), minimal inference latency ($< 30\ ms$).
- Environment: Real-time video feed, variable ambient lighting, dynamic facial angles/poses, occlusion (glasses, masks).
- Actuators: Screen display showing bounding boxes, predicted emotion text labels, emotion intensity logs, or triggers for downstream UI actions.
- Sensors: High-definition RGB digital camera, webcam, or video frame stream buffer.

(d) Agent Classification & Justification

- Classification: Model-Based Reflex Agent.
- Justification: Simple reflex mapping from single pixel frames is insufficient due to lighting variations, angles, and facial dynamics. The agent uses an internal state representation (a model of human facial geometry, spatial landmark mappings, and learned features) to evaluate emotion states from current visual percepts.

**Question 3: Medical Diagnosis Expert System Shell**

(a) Type of Chaining Used & Justification

- Type of Chaining: Forward Chaining (Data-driven / Antecedent-to-Consequent reasoning).
- Justification: The inference engine starts with known data facts (fever = yes, cough = yes, body_ache = yes) and matches them against the IF conditions of rules to derive new conclusions iteratively until a final diagnosis is established:
    1. Given fever = yes and cough = yes → Condition for R1 is met → Infers new fact: suspect = flu.
    2. suspect = flu and existing body_ache = yes → Condition for R2 is met → Infers final diagnosis: diagnosis = Influenza.

(b) Expert System Shell Components Illustrated

1. Knowledge Base (KB): Contains domain-specific production rules (R1, R2, R3).
2. Inference Engine: Applies the match-resolve-execute cycle (Forward Chaining pattern) using given facts to fire matching rules (R1 then R2).
3. Explanation Facility: Generates human-readable traces explaining *how* the conclusion was reached (*"Diagnosis = Influenza because fever and cough indicated flu..."*).
4. Working Memory / Fact Base: Holds working input facts (fever=yes, cough=yes, body_ache=yes) and newly asserted intermediate facts (suspect=flu).

(c) Knowledge Acquisition for Adding a New Dengue Rule

Knowledge Acquisition translates domain expert (physician) knowledge into rule representations in the shell:

1. Knowledge Elicitation: Acquire diagnostic criteria for Dengue from medical experts (e.g., high fever + severe joint pain + skin rash).
2. Rule Formalization: Translate the elicited knowledge into the standard IF-THEN format:

$$IF\ (fever = yes\ AND\ joint\_pain = yes\ AND\ rash = yes) \Longrightarrow THEN\ (diagnosis = Dengue)$$

3. Integration & Validation: Insert the new rule into the Rule Base (R4), verify syntax, and run consistency tests against existing facts to avoid logical contradictions with R3 (measles).

(d) Analysis of Backward Chaining

- Can Backward Chaining reach the same diagnosis? Yes.
- Reasoning: Backward chaining is Goal-driven. It starts with a hypothesis (e.g., Goal: diagnosis = Influenza) and works backward:
    1. To prove diagnosis = Influenza, check R2 → Requires suspect = flu and body_ache = yes.
    2. body_ache = yes is verified in Working Memory.
    3. To prove suspect = flu, check R1 → Requires fever = yes and cough = yes.
    4. Both fever = yes and cough = yes are verified in Working Memory.
    5. Goal confirmed: diagnosis = Influenza.

**Question 4: Autonomous Drone for Disaster Detection (Edge AI)**

(a) PEAS Description

- Performance Measure: Accurate disaster detection rate, collision-free flight, low system response latency, optimal battery usage/flight time, minimal communication overhead.
- Environment: Dynamic 3D air space, harsh weather conditions, changing smoke/debris, GPS-degraded zones, obstacles (trees, damaged buildings).
- Actuators: Motor speed controllers (rotors/propellers), gimbal stabilization camera motors, emergency parachute, wireless transmitter/telemetry module.
- Sensors: Onboard RGB/Thermal Cameras, LiDAR, Ultrasonic distance sensors, Inertial Measurement Unit (IMU), GPS.

(b) Agent Classification & Justification

- Classification: Utility-Based Agent.
- Justification: A disaster drone operates under multiple conflicting constraints (battery life vs. search coverage, fast navigation vs. safe obstacle avoidance). A pure goal-based agent only seeks binary success; a utility-based agent evaluates a real-valued utility function to trade off flight safety, detection speed, coverage area, and energy consumption to pick optimal paths.

(c) Edge vs. Cloud Architecture & Edge Justification

| Component | Execution Location | Primary Reason |
|---|---|---|
| Object/Disaster Visual Classifier | Edge (Onboard AI Accelerator) | Ultra-low latency inference, high throughput. |
| Obstacle Avoidance & Collision Prevention | Edge (Microcontroller / Edge SoC) | Real-time safety guarantees ( $\leq 10\ ms$ response). |
| FLIGHT Motor Controllers / IMU Processing | Edge (Flight Controller) | Real-time critical closed-loop stability. |
| Global Path Planning / Fleet Analytics | Cloud | Computationally heavy global spatial mapping. |
| Long-term Video Logs Archiving | Cloud | Storage capacity scaling. |

- Justification for On-Edge Processing:
    1. Zero-Latency Safety Criticality: Obstacle detection and avoidance require immediate response times ($< 10\ ms$) to prevent crashes; network round-trips ( $100 \ -- \ 500\ ms$) would cause catastrophic collisions.
    2. Network Autonomy / Connectivity Losses: Disaster zones frequently lack cellular or satellite infrastructure. Edge processing enables uninterrupted operational capability without continuous cloud connection.

(d) Control Architecture Classification & Justification

- Classification: Hybrid Control Architecture (e.g., Subsumption / Layered Architecture).
- Justification: The drone incorporates both behaviors:
  Reactive Layer (Lower Level): Directly maps real-time LiDAR/sensor inputs to immediate flight corrections for obstacle avoidance without higher-level deliberative delays.
  Deliberative Layer (Higher Level): Computes macro-level spatial path planning, mission objectives, and goal evaluation.

**Question 5: Deepfake & Face Authenticity Verification System**

(a) Input Features Identified from Logs

1. Blink Rate (per 10s): Physiological feature monitoring eye movement patterns and biological blink frequencies over time.

2. Texture / Artifact Score: Spatial frequency visual feature evaluating pixel anomalies, boundary blurring, warping errors, or GAN/diffusion reconstruction artifacts.

(b) Underlying Models / Techniques Used

- Spatial Feature Extraction: Convolutional Neural Networks (CNNs) like EfficientNet or ResNet to compute frame-level Texture/Artifact Score.
- Temporal / Sequence Analysis: Recurrent Neural Networks (RNN / LSTM) or 3D Convolutional Networks (3D-CNN) to evaluate temporal anomalies (Blink Rate).
- Binary Classifier Head: Dense Layer with Sigmoid activation outputting the Fake Confidence (%).

(c) Derivation of Final Decision from Frame Scores

- Aggregation Technique: Temporal Majority Voting Logic across frame predictions.
- Step-by-step Math/Logic:
    1. Each frame undergoes individual classification based on a threshold ( $Confidence\ threshold \geq 50\%$ classified as Fake).
    2. *Frame Results:*

       Frame 1: 8% → Real

       Frame 2: 74% → Fake

       Frame 3: 91% → Fake

       Frame 4: 5% → Real

       Frame 5: 68% → Fake
    3. *Tally:* Total Frames = 5. Fake Flags = 3, Real Flags = 2.
    4. Since $Fake\ Count\ (3) > Real\ Count\ (2)$, the system outputs an overall video decision of Fake.

(d) Enhancing Reliability via Swarm / Ensemble Approach

- Method: Multi-Expert Ensemble / Stacking Classifier Approach.
- Mechanism: Rather than relying on a single model pipeline, combine multiple diverse specialized models:
    - *Expert Model 1:* Physiological Analysis (Blink rate, heart rate / rPPG signals).
    - *Expert Model 2:* Spatial Boundary & Frequency Artifact Detector (Fourier transform analysis).
    - *Expert Model 3:* Audio-Visual Lip Synchronization Model.
- Benefit: Outliers in single frames (e.g., natural non-blinking in Frame 2 or sudden compression artifacts) will be voted down by parallel independent domain experts, preventing false positives and improving detection robustness against modern spoofing attacks.